

UNIVERSIDAD PRIVADA DE TACNA
FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto de Unidad 1 — DespliegaUML

Diagramador de componentes y despliegue de software con validación de reglas de consistencia y generación de plan de despliegue

Curso: Calidad y Pruebas de Software (SI-784)

Docente: CUADROS QUIROGA, Patrick Jose

Ciclo 7 · 2026-II

Integrantes:

RODRIGUEZ CARDENAS, Patrick — 2022075751

RIOS COHAILA, Nicole — 2022075745

Tacna – Perú · 2026

1. Título

DespliegaUML: herramienta para modelar diagramas de componentes y despliegue de software, validar su consistencia mediante reglas y generar el plan de despliegue correspondiente.

El nombre agrupa las dos capacidades centrales de la herramienta: «Despliega» por la generación del plan de despliegue, y «UML» por el modelado en el lenguaje estándar de la disciplina.

2. Problema detallado con fuentes

2.1. Planteamiento del problema

El modelado de la arquitectura de software —el diagrama de componentes y el diagrama de despliegue según UML 2— se realiza habitualmente con herramientas genéricas de dibujo. Estas herramientas permiten representar las cajas y las flechas, pero no verifican que lo representado sea coherente. Un componente puede quedar enlazado a un elemento incorrecto, un nodo de despliegue puede señalar artefactos que no existen, o pueden formarse ciclos de ensamblaje no permitidos, sin que el autor lo advierta hasta la fase de implementación, cuando el costo de corregirlo ya es alto.

A esta limitación se suma una segunda: las herramientas de dibujo no producen ningún artefacto de despliegue. El diagrama queda como una imagen; el equipo tiene que redactar a mano el plan de despliegue, y ese plan se desincroniza del modelo en cuanto alguien modifica un componente.

2.2. Evidencia en la actividad regional de desarrollo de software

Radio Uno, medio de comunicación de alcance regional de Tacna, difunde de forma constante noticias sobre servicios y sistemas locales, lo que evidencia que en la región existe actividad sostenida de desarrollo de software y, con ella, la necesidad de prácticas de despliegue ordenadas. Un entorno que produce software de forma continua necesita, antes que nada, poder verificar que lo que despliega es lo que cree estar desplegando.

Fuente: <https://radiouno.pe/>

2.3. Evidencia desde la validación del problema como criterio de proyecto

ProblemHunt es una plataforma donde personas reales publican los problemas que tienen y pagan por su solución; su criterio de fondo es que un problema no es oportunidad hasta que alguien está dispuesto a pagar por resolverlo. Aplicando ese criterio al modelado de arquitectura, la disposición a pagar existe precisamente en los equipos que gestionan despliegues: una validación automatizada del modelo evita retrabajo, y el retrabajo es un costo directamente atribuible al despliegue fallido.

Fuente: <https://problemhunt.pro/>

2.4. Vacío detectado en los antecedentes de la facultad

El repositorio institucional de la Facultad de Ingeniería muestra antecedentes de proyectos de gestión y desarrollo de software, pero ninguno aborda la validación automática de diagramas de componentes y de despliegue. Este proyecto propone cubrir ese vacío, usando lo previo únicamente como base de estudio y no como copia: la aportación es el motor de reglas de consistencia y la generación del plan, no el redibujo de un diagrama.

Fuente: <https://github.com/UPT-FAING-EPIS/>

2.5. Marco de referencia para la trazabilidad documental

RENATI es el Registro Nacional de Trabajos de Investigación de la SUNEDU, creado para resguardar la investigación universitaria, preservar el acceso digital a los trabajos y asegurar la disponibilidad de sus metadatos. Se cita aquí como marco de la práctica documental del curso: el proyecto debe producir documentación técnica trazable, no sólo código ejecutable.

Fuente: <https://renati.sunedu.gob.pe/>

3. Objetivos de investigación (medibles)

Los objetivos de investigación responden a la pregunta de investigación del proyecto: ¿el conjunto de reglas propuesto detecta realmente los errores de modelado que motivan el problema, y el plan que produce son realmente utilizable para desplegar?

#	Objetivo de investigación	Cómo se mide
OI-1	Determinar en qué medida un conjunto de reglas de consistencia detecta errores de modelado en diagramas de componentes y despliegue.	Por la eficacia de detección: porcentaje de casos de prueba negativos que hacen disparar su regla correspondiente, con casos positivos que verifican la ausencia de falsos positivos. Meta: 100 % de eficacia y 0 % de falsos positivos.
OI-2	Evaluar la claridad del plan de despliegue generado a partir del modelo validado.	Por la cobertura del plan: porcentaje de nodos y artefactos del modelo que aparecen en el plan generado, sin referencias pendientes ni supuestos sin declarar. Meta: cobertura 100 %.

4. Objetivos de solución (medibles)

Los objetivos de solución traducen los objetivos de investigación en entregables concretos del proyecto.

#	Objetivo de solución	Cómo se mide
OS-1	Implementar un editor de diagramas de componentes y de despliegue que permita crear, editar y eliminar cada tipo de elemento del modelo.	Cobertura funcional: cada elemento del modelo (componente, puerto, conector, nodo, artefacto, realización y camino) debe poder crearse, modificarse y eliminarse desde la interfaz.
OS-2	Implementar un motor de validación de reglas de consistencia.	Cada regla definida debe tener al menos un caso de prueba positivo (un modelo que no la dispare) y uno negativo (un modelo que sí la dispare). Meta: cobertura del 100 % de las reglas del catálogo.
OS-3	Generar el plan de despliegue a partir del modelo validado.	El plan debe generarse para todo modelo válido y bloquearse, con reporte de errores, para todo modelo inválido. Se verifica con ambos casos.
OS-4	Verificar la lógica del modelo y la validación con pruebas automatizadas de sus reglas.	Todas las pruebas automatizadas deben ejecutarse y pasar. Se registra la tasa de éxito y se reportan los defectos detectados.

5. Calendario de entregas y estado de avance

El proyecto de unidad se desarrolla en seis semanas con el siguiente calendario de entregables:

Semana	Entregable	Estado
1	Título, problema detallado con fuentes, objetivos de investigación y solución medibles	ENTREGADO — este documento
2	FD01 Informe de Factibilidad · FD02 Informe de Visión	ENTREGADO
3	FD03 Informe de Especificación de Requerimientos de Software (SRS) · FD04 Informe de Arquitectura de Software (SAD)	ENTREGADO
4, 5, 6	Aplicación desplegada en nube o servicio público, mediante automatizaciones que generan adicionalmente los diagramas y manuales técnicos desde un repositorio git	APLICACIÓN Y CI/CD ENTREGADOS. El pipeline genera los diagramas y el manual técnico en cada push. Falta un clic de un administrador de la organización para activar la rama main en el entorno de Pages.

Repositorio del proyecto: <https://github.com/UPT-FAING-EPIS/proyecto-si784-2026-ii-cobijita>

La aplicación (`app/index.html`) se ejecuta sin instalación: se abre con doble clic. Las 64 pruebas automatizadas se ejecutan con: `node app/tests/casos.js`